1 Introduction

In this laboratory work, we observe Arduino circuits and buttons, with a problem known as "bouncing". This issue is due to mechanical aspects leading to unexpected open/close transitions. We will do experiments to observe this effect via an oscilloscope and apply a debouncing coding via Arduino to prevent the bouncing. This report will include our observations and insights from the experiments.

2 Experiments

In this laboratory session, we will hold two different experiments. First experiment will be about the how to build and use circuit with basic button while analyzing the response of the button and second experiment will be about how to debounce it. Then we will add a counter to see how many times the button is pressed and print the counter values in binaries.

2.1 Experiment 1: Bouncing

In our first experiment, we will construct a basic circuit with button as shown in the Figure 1. Afterwards, we will observe the bouncing effect of the button.

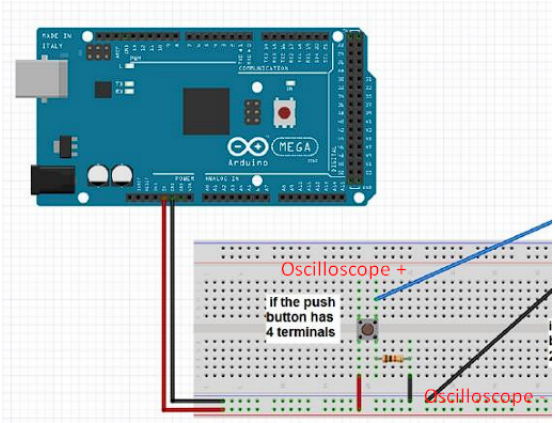


Figure 1: Experiment 1 schematic of circuit.

In this experiment we used these components listed below.

- 1X Arduino
- 1X 10kΩ resistor
- 1X Push button
- 2X Jumper cables
- 1X Breadboard

We have constructed the circuit shown in Figure 1 as shown below in Figure 2 and connected +5V and GND from Arduino then we have connected oscilloscope to GND and end side of the button in order to observe the behavior of button on the oscilloscope.

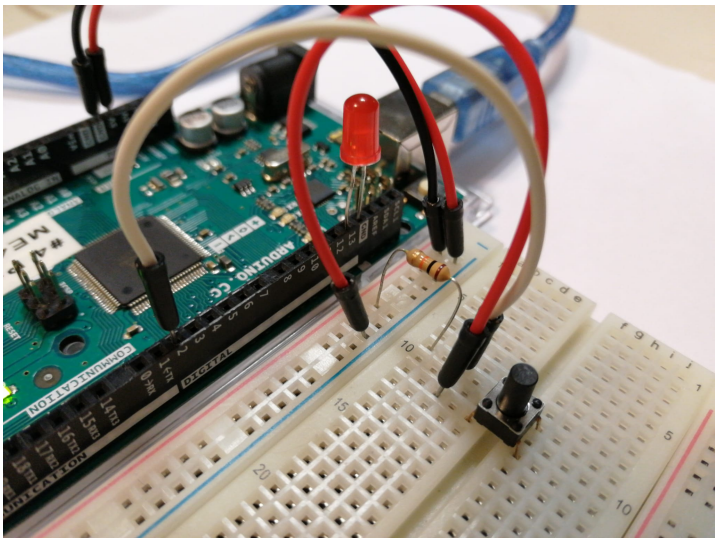


Figure 2: Circuit shown in the Figure 1 is constructed on the breadboard.

In our case, buttons did not actually cause any bouncing effects at all because "bouncing" was seen on the older versions of buttons which are not capable to handle bouncing effect. But now our buttons can handle it, so we had to press with more power than usual, and tried many times to catch the data.

After some tries we managed to catch the representation of the behaviour of button on the oscilloscopes' screen as shown in the Figure 3.

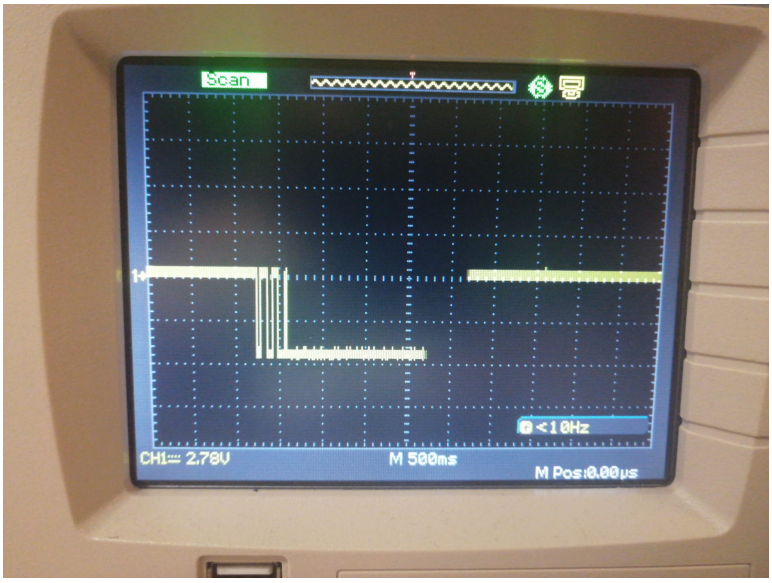


Figure 3: Buttons' behaviour shown on the oscilloscope.

2.2 Experiment 2: Debouncing

In the previous experiment we had observed "bouncing" effect and understand that effect happens in a really short time so that means we can set an threshold for activation and deactivation with Arduino.

In our second experiment we will try to set a threshold with Arduino so we can be sure the button is pressed. In order to set that we have implemented the code shown below. If the button is pressed purposely, LED's will be ON or OFF.

```
1
2 / constants won     t change . They     re used here to set pin numbers :
3 const int buttonPin = 2; // the number of the pushbutton pin
4 const int ledPin = 13; // the number of the LED pin
5
6 // Variables will change :
7 int ledState = HIGH ; // the current state of the output pin
8 int buttonState ; // the current reading from the input pin
9 int lastButtonState = LOW ; // the previous reading from the input pin
10
11 // the following variables are unsigned longs because the time , measured in
12 // milliseconds , will quickly become a bigger number than can be stored in an int.
13 unsigned long lastDebounceTime = 0; // the last time the output pin was toggled
14 unsigned long debounceDelay = 50; // the debounce time ; increase if the output fli
15
16 void setup () {
17 pinMode ( buttonPin , INPUT );
18 pinMode ( ledPin , OUTPUT );
19
20 // set initial LED state
21 digitalWrite ( ledPin , ledState );
22 }
23
24 void loop () {
25 // read the state of the switch into a local variable :
26 int reading = digitalRead ( buttonPin );
27
28 // check to see if you just pressed the button
29 // (i.e. the input went from LOW to HIGH ) , and you have waited long enough
30 // since the last press to ignore any noise :
31
32 // If the switch changed , due to noise or pressing :
33 if ( reading != lastButtonState ) {
34 // reset the debouncing timer
35 lastDebounceTime = millis ();
36 }
37
38 if (( millis () - lastDebounceTime ) > debounceDelay ) {
39 // whatever the reading is at , it is been there for longer than the debounce
40 // delay , so take it as the actual current state :
41
42 // if the button state has changed :
43 if ( reading != buttonState ) {
44 buttonState = reading ;
45
```

```
46 // only toggle the LED if the new button state is HIGH
47 if ( buttonState == HIGH ) {
48 ledState = ! ledState ;
4
49 }
50 }
51 }
52
53 // set the LED:
54 digitalWrite ( ledPin , ledState );
55
56 // save the reading . Next time through the loop , it will be the lastButtonState :
57 lastButtonState = reading ;
58 }
```

With that code shown above, the LED will be turned ON or OFF without any problems or experiencing any bouncing.

2.2.1 Adding Counter and Printing Counter Values in Binary

Now we will edit the code previously given and add a counter to count how many times the button is pressed. Then we will print the counter value in binary values.

```
1
2 // constants will not change . They are used here to set pin numbers :
3 const int buttonPin = 2; // the number of the pushbutton pin
4 const int ledPin = 13; // the number of the LED pin
5
6 // Variables will change :
7 int ledState = HIGH ; // the current state of the output pin
8 int buttonState ; // the current reading from the input pin
9 int lastButtonState = LOW ; // the previous reading from the input pin
10 int number = 0;
11
12 // the following variables are unsigned longs because the time , measured in
13 // milliseconds , will quickly become a bigger number than can be stored in an int.
14 unsigned long lastDebounceTime = 0; // the last time the output pin was toggled
15 unsigned long debounceDelay = 50; // the debounce time ; increase if the output
flickers
16
17 void setup () {
18 pinMode ( buttonPin , INPUT );
19 pinMode ( ledPin , OUTPUT );
20
21 // set initial LED state
22 digitalWrite ( ledPin , ledState );
23 Serial . begin (9600);
24 }
25
26 void loop () {
27 // read the state of the switch into a local variable :
28 int reading = digitalRead ( buttonPin );
29
30 // check to see if you just pressed the button
31 // (i.e. the input went from LOW to HIGH ) , and you have waited long enough
```

```
32 // since the last press to ignore any noise :
33
34 // If the switch changed , due to noise or pressing :
35 if ( reading != lastButtonState ) {
36 // reset the debouncing timer
37 lastDebounceTime = millis ();
38 }
39
40 if (( millis () - lastDebounceTime ) > debounceDelay ) {
41 // whatever the reading is at , it is been there for longer than the debounce
42 // delay , so take it as the actual current state :
43
44 // if the button state has changed :
45 if ( reading != buttonState ) {
46 buttonState = reading ;
47 number = number + 1;
48
6
49 // only toggle the LED if the new button state is HIGH
50 if ( buttonState == HIGH ) {
51 if ( number >= 15){
52 number = 0;
53 Serial . println ( number , BIN );
54 } else {
55 Serial . println ( number , BIN );
56 }
57
58 }
59 }
60 }
61
62 // set the LED:
63 digitalWrite ( ledPin , ledState );
64
65 // save the reading . Next time through the loop , it will be the lastButtonState :
66 lastButtonState = reading ;
67 }
```

After implementing that code shown above, we have tested the code with circuit we observed that code runs perfectly without any error.

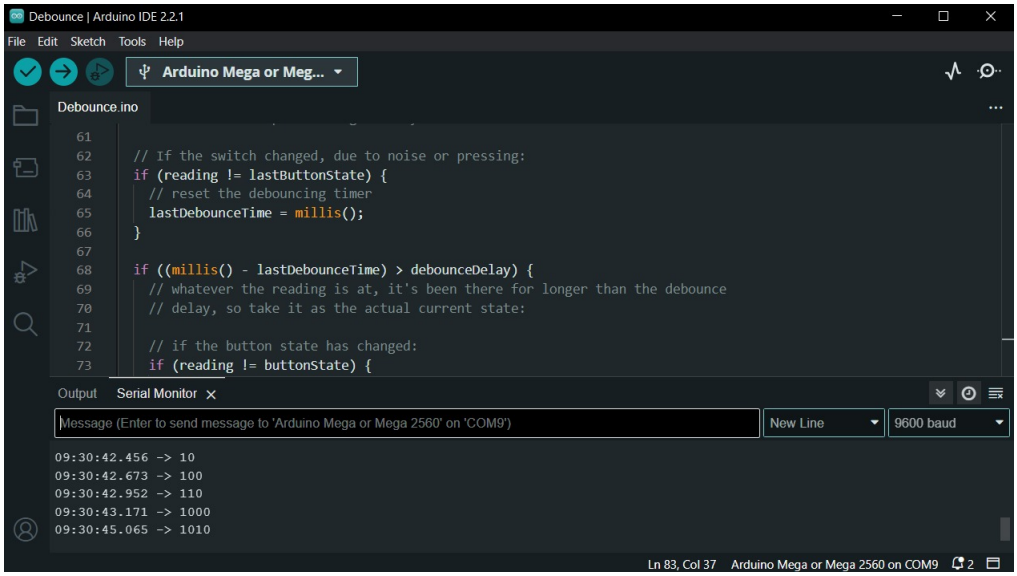


Figure 4: Arduino output on console.

3 Conclusion

In conclusion, our laboratory experiments provided understanding of the 'bouncing' effect with buttons and debouncing with Arduino. Our experiments with Arduino circuits showed the importance of software solutions like debouncing algorithms to be sure about the readings from analogs. The addition of a binary counter presented the application of digital logic and integrating software problem-solving approaches. Overall, these experiments have provided knowledge about electronic circuit behaviors and role of the programming in resolving hardware problems.